

# Erratum and Revisions of The Hash Function MOZI

MOZI submission team

September 28, 2026

## Abstract

This document provides errata and revisions to the specification of MOZI, a hash function family submitted to the NGCC program. The errata correct inaccuracies and ambiguities in the submitted specification and the revisions add a domain-separation mechanism in the initialization phase for all variants. A complete list of changes is given in the table below. Accordingly, the reference and optimized implementations have been updated, and all known-answer test (KAT) vectors have been regenerated for all variants of the family.

Table 1: Summary of changes to the submitted specification.

| # | Location      | Change                                                                                                                                                                                     |
|---|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Section 2.3   | In Algorithm 1 and 2, a domain encoding is added in the initialization phase. Accordingly, the initialization in Figures 1 and 2 is updated to include the domain encoding.                |
| 2 | Section 2.4.2 | The matrix for $\alpha$ in the MDS matrix is replaced by its transpose, so that it is consistent with the definition given in Algorithm 4.                                                 |
| 3 | Section 2.5   | A dedicated section on domain separation Section 2.5 is added, specifying distinct domain encodings applied in the initialization phase for each variant of the MOZI hash function family. |

## 1 Addition of Domain separation

We use the last four bits in  $IV$  to make the domain separation for MOZI. That is, in the initialization phase, the initial state is assigned by concatenating 2044 bits of zeros with the 4-bit domain encoding, which is located in the least significant

4 bits in capacity. The domain encoding for different versions is listed in Table 5. Different versions have different domain encode numbers fed in the last four bits of IV. For example, the input IV value is 2044 bit zeros, concatenating 4 bits 0001 for MOZI-384 in the initialization phase, while for MOZI-384-XOF, the four bit number is 1001.

Table 5: Domain encoding for different hash and XOF versions

| Versions      | MOZI-384 | MOZI-512 | MOZI-768 | MOZI-1024 | MOZI-384-XOF | MOZI-512-XOF |
|---------------|----------|----------|----------|-----------|--------------|--------------|
| Domain values | 0001     | 0011     | 0100     | 0101      | 1001         | 1011         |

## 2 Sponge function and Feed-forward Sponge

The revised specification of Section 2.3 is reproduced below. Compared with the submitted version, the initialization now incorporates the domain encoding defined in Sect. 1.

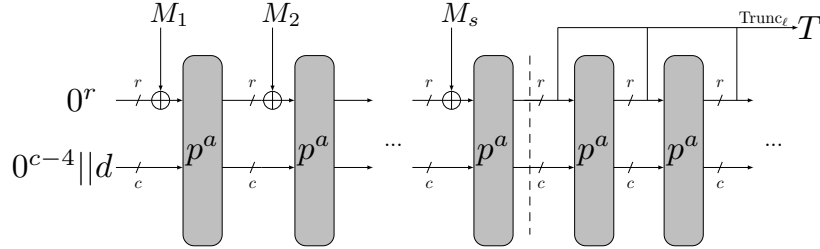


Figure 1: Sponge function

**Sponge function**  $\text{SPONGE}_{h,r,a}(M, \ell)$ . The Sponge structure is given in Figure 1 and  $\text{SPONGE}_{h,r,a}(M, \ell)$  is specified in Algorithm 1. In the initialization phase, four-bit domain encoding number  $d$  is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 1.

---

**Algorithm 1:** The Sponge function  $\text{SPONGE}_{h,r,a}(M, \ell)$ 


---

**Input:** Message  $M \in \{0, 1\}^*$ , output bitsize  $\ell \leq h$  or arbitrary if  $h = 0$   
**Output:** Digest  $T \in \{0, 1\}^\ell$   
 /\* Initialization, 4-bit domain encoding  $d$  is specified in Table 5 for different  $\ell$  \*/  
 1  $\mathcal{I} \leftarrow 0^{2044} \| d$   
 /\* Absorbing \*/  
 2  $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$ , where the bitsize of  $M_i$  ( $1 \leq i \leq s$ ) is  $r$  bits  
 3 **for**  $i = 1$  **to**  $s$  **do**  
 4      $\mathcal{I} \leftarrow p^a(\mathcal{I} \oplus (M_i \| 0^{2048-r}))$   
 /\* Squeezing \*/  
 5  $\lambda \leftarrow \lfloor \frac{\ell}{r} \rfloor$   
 6  $Z \leftarrow \text{MSB}_r(\mathcal{I})$   
 7 **for**  $i = 1$  **to**  $\lambda$  **do**  
 8      $\mathcal{I} \leftarrow p^a(\mathcal{I})$   
 9      $Z \leftarrow Z \| \text{MSB}_r(\mathcal{I})$   
 10 **return**  $T = \text{MSB}_\ell(Z)$

---

**Feed-forward Sponge Function  $\text{F-SPONGE}_{h,r,b}(M)$ .** The feed-forward sponge construction [2, 1] is given in Figure 2, and  $\text{F-SPONGE}_{1024,960,24}(M)$  is specified in Algorithm 2. In the initialization phase, four-bit domain encoding number  $d$  is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 1.

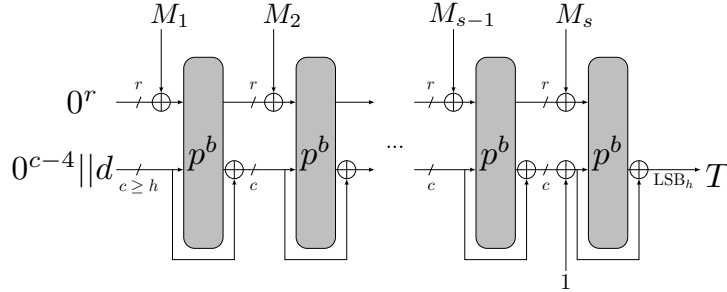


Figure 2: Feed-forward Sponge function

---

**Algorithm 2:** The feed-forward Sponge function  $\text{F-SPONGE}_{h,r,b}(M)$ 


---

**Input:** Message  $M \in \{0,1\}^*$   
**Output:** Digest  $T \in \{0,1\}^h$   
 /\* Initialization, 4-bit domain encoding  $d$  is specified in Table 5 for different  $h$  \*/  
 1  $\mathcal{I} \leftarrow 0^{2044} \| d$   
 /\* Absorbing \*/  
 2  $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$ , where the bitsize of  $M_i$  ( $1 \leq i \leq s$ ) is  $r$  bits  
 3 **for**  $i = 1$  **to**  $s$  **do**  
 4     **if**  $i \neq s$  **then**  
 5          $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 0^c)$   
 6     **else**  
 7          $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 0^{c-1} \| 1)$   
 8      $\mathcal{I} \leftarrow (0^r \| \text{LSB}_c(\mathcal{I})) \oplus p^b(\mathcal{I})$   
 /\* Squeezing \*/  
 9  $T \leftarrow \text{LSB}_h(\mathcal{I})$   
 10 **return**  $T$

---

### 3 Transposition of the $\alpha$ Matrix

In Section 2.4.2, the matrix displayed for  $\alpha$  is replaced by its transpose:

$$\alpha : \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix} \mapsto \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

i.e.,  $\alpha(x_0, x_1, x_2, x_3) = (x_3, x_0 \oplus x_3, x_1, x_2)$ . The revised matrix now is consistent with the MUL routine of Algorithm 4 and the submitted implementations.

### 4 Package Update

The reference and optimized implementations have been updated to reflect the corrections above. In particular, the position of the final marker of the feed-forward sponge was inconsistent between the algorithm description and the implementations, and the implementations have been corrected accordingly, and all known-answer test vector files have been regenerated for each instance.

### References

- [1] Chun Guo, Kai Hu, Yanhong Fan, Yong Fu, and Meiqin Wang. Adding feeding forward back to the sponge construction. Cryptology ePrint Archive, Paper 2025/1006, 2025.

- [2] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. Cryptology ePrint Archive, Paper 2025/963, 2025.